



System Design Tutorial

Phase 1 — Core Components, Explained from First Principles

A complete, beginner-to-FAANG guide to designing large-scale systems

Volume 1 of 4 | Client/Server · DNS · Load Balancers · Databases · Caching · CDNs · Queues
· APIs · Proxies · Storage · Networking

Table of Contents

Welcome — How to Use This Tutorial	7
How each component is taught	7
1. Client & Server Architecture	8
What is it?	8
Why does it exist?	8
How does it work internally?	8
Thick vs thin clients	8
When should you use it?	9
Real-world examples	9
2. DNS — The Domain Name System	10
What is it?	10
Why does it exist?	10
How does it work internally?	10
Common record types you should know	10
DNS as a load-balancing and routing tool	11
When should you use it?	11
Real-world examples	11
3. Load Balancers	12
What is it?	12
Why does it exist?	12
How does it work internally?	12
Layer 4 vs Layer 7	12
Balancing algorithms	13
Consistent hashing — the one to understand deeply	13
When should you use it?	14
Real-world examples	14
4. Web Servers vs Application Servers	15
What is it?	15

Why does it exist?	15
How does it work internally?	15
When should you use it?	15
Real-world examples	15

5. Databases — SQL vs NoSQL 17

What is it?	17
Why does it exist?	17
SQL (relational) databases	17
NoSQL databases	17
How does it work internally? (indexing & storage)	18
When should you use which?	18
Real-world examples	19

6. Caching 20

What is it?	20
Why does it exist?	20
How does it work internally? (cache levels)	20
Caching strategies	20
Eviction policies	21
When should you use it?	21
Real-world examples	21

7. Content Delivery Networks (CDN) 22

What is it?	22
Why does it exist?	22
How does it work internally?	22
When should you use it?	22
Real-world examples	22

8. Message Queues & Event Streaming 24

What is it?	24
Why does it exist?	24
How does it work internally?	24

Two messaging patterns	24
Message queues vs event streaming	25
When should you use it?	25
Real-world examples	25
9. APIs — REST, GraphQL, gRPC	26
What is it?	26
Why does it exist?	26
The three main styles	26
REST	26
GraphQL	26
gRPC	26
Choosing between them	27
When should you use it?	27
Real-world examples	27
10. Proxies — Forward vs Reverse	28
What is it?	28
Forward proxy	28
Reverse proxy	28
When should you use it?	28
Real-world examples	28
11. Storage Systems — Block, File, Object	30
What is it?	30
The three types	30
Block storage	30
File storage	30
Object storage	30
How does it work internally? (distributed storage)	30
When should you use which?	31
Real-world examples	31
12. Networking Basics	32
What is it?	32

TCP vs UDP	32
HTTP and HTTPS	32
Keeping a live connection: polling, long polling, and WebSockets	32
When should you use which?	33
Real-world examples	33

Phase 1 — Key Takeaways

34

Welcome — How to Use This Tutorial

If you have never designed a system before, you are in exactly the right place. We will start from zero and build up, one brick at a time.

System design has a reputation for being intimidating. People throw around words like *sharding*, *eventual consistency*, and *back-pressure* as if everyone was born knowing them. They were not. Every one of those ideas is simple once someone explains it slowly, with a picture and an everyday analogy. That is the promise of this guide.

Think of building a software system like building a city. A city needs roads (networking), addresses so the post office can find houses (DNS), traffic police directing cars (load balancers), warehouses to store goods (databases and storage), corner shops that keep popular items close to people (caches), a postal service that delivers packages reliably even when one route is blocked (message queues), and rules about who is allowed into which building (security). In this first volume we tour each of these pieces on its own. In later volumes we learn the principles that hold them together, then practise designing whole 'cities' from scratch.

How each component is taught

For every component you will see the same five questions answered, in the same order, so the material becomes predictable and easy to revise:

- **What is it?** — a plain-English definition you could explain to a friend.
- **Why does it exist?** — the real problem it was invented to solve.
- **How does it work internally?** — what is actually happening under the hood.
- **When should you use it?** — and, just as important, when you should *not*.
- **Real-world examples** — the actual products and companies that use it.

How to study this

- Read each component slowly once. Do not try to memorise — aim to *understand*.
- Re-draw each ASCII diagram by hand. The act of drawing cements the idea.
- After each component, close the book and explain it out loud in one minute.
- The 'Key Takeaways' box at the end of the phase is your revision sheet.

1. Client & Server Architecture

What is it?

A **client** is the program that asks for something. A **server** is the program that answers. Your phone's Instagram app is a client; the computers in Meta's data centre that send back your photos are servers. That is the entire idea: one side requests, the other side responds.

Analogy: a client is a customer at a restaurant; the server (literally a waiter) takes the order, the kitchen prepares it, and the food comes back. The customer never enters the kitchen.

Why does it exist?

In the early days, a single computer did everything for one user sitting in front of it. But software became useful precisely because many people wanted the same data and features — your bank balance, your email, a shared document. You cannot put the bank's entire database on every customer's laptop. So we split responsibilities: lightweight clients on millions of devices, and powerful shared servers in a central place that everyone connects to. This separation lets us update the logic in one place, keep data consistent, and secure the valuable parts behind locked doors.

How does it work internally?

A client and server talk over a **network** using an agreed **protocol** (a shared language). The most common one on the web is HTTP. The flow looks like this:



Figure 1.1 - The request/response cycle between a client and a server.

The key mental model: every interaction is a **request** followed by a **response**. The client opens a connection, sends a request ("GET me the home page"), the server does work and sends back a response (the HTML, or some data), and — in classic HTTP — the connection can then close. The server is usually **stateless** for web traffic, meaning it does not remember you between requests; your identity travels with each request inside a token or cookie. Statelessness is what later lets us run many identical servers behind a load balancer.

Thick vs thin clients

A **thin client** does very little and leans on the server for almost everything (an old web page). A **thick (or rich) client** does a lot of work itself (a mobile game, or a single-page web app that holds logic in JavaScript). Modern apps sit somewhere in between, balancing speed on the

device against control and consistency on the server.

When should you use it?

Practically every networked application is client-server, so the real question in an interview is *how* you split the work. Put on the server anything that must be trusted (payment logic, permissions), anything shared across users (the source-of-truth data), and anything too heavy for a phone. Put on the client anything that improves responsiveness (input validation, optimistic UI updates) — but never trust the client for security, because a determined user controls their own device.

Real-world examples

- **Web:** Chrome (client) talking to Google's servers over HTTP.
- **Mobile:** the Uber app (client) talking to Uber's backend services.
- **Email:** Outlook (client) talking to a mail server using SMTP/IMAP.
- **Games:** Fortnite (client) talking to game servers that hold the authoritative match state.

2. DNS — The Domain Name System

What is it?

DNS is the **phone book of the internet**. Humans remember names like `google.com`; computers route traffic using numeric addresses like `142.250.72.14` (an IP address). DNS is the system that translates the friendly name into the numeric address so your request can find its destination.

Why does it exist?

Imagine if you had to memorise the IP address of every website you visit, and those addresses changed whenever a company moved servers. Unworkable. DNS gives us stable, human-friendly names and lets the underlying machines move around freely behind the scenes. It also enables powerful tricks: returning a *different* server's address depending on where you are in the world, or spreading load across many machines.

How does it work internally?

When you type a name, your computer asks a chain of helpers until someone knows the answer. This is called **resolution**:

```
You type:  shop.example.com

[Your computer] --> [Recursive Resolver]  (usually your ISP or 8.8.8.8)
      |
      1. ask the ROOT server: "who handles .com?"
      v
      [Root server] --> "ask the .com TLD server"
      |
      2. ask the .COM TLD server: "who handles example.com?"
      v
      [.com TLD server] --> "ask example.com's nameserver"
      |
      3. ask EXAMPLE.COM's authoritative nameserver
      v
      [Authoritative NS] --> "shop.example.com = 93.184.216.34"
      |
      v
      Resolver caches the answer and returns it to you
```

Figure 2.1 - A recursive DNS lookup walks down a hierarchy: Root -> TLD -> Authoritative.

Two ideas make this fast despite all those hops. First, **caching**: once a resolver learns an answer it remembers it for a while (the **TTL**, or time-to-live, set by the domain owner), so most lookups never reach the root. Second, the hierarchy is shared by the whole world, so answers are usually one cache-hit away.

Common record types you should know

Record	Purpose	Example value
A	Maps a name to an IPv4 address	93.184.216.34
AAAA	Maps a name to an IPv6 address	2606:2800:220:1::1
CNAME	Alias one name to another name	www -> example.com
MX	Where to deliver email for the domain	mail.example.com
NS	Which nameservers are authoritative	ns1.example.com
TXT	Arbitrary text (e.g. domain verification)	v=spf1 ...

DNS as a load-balancing and routing tool

Because the domain owner controls the answers, DNS becomes a steering wheel. **Round-robin DNS** returns several IPs and clients pick one, spreading load. **GeoDNS** returns the address of the nearest data centre, cutting latency. **Health-aware DNS** stops returning the address of a server that has failed. This is why DNS is not just an address book — it is the first layer of traffic management in large systems.

When should you use it?

You always use DNS — the design choice is how aggressively. Use short TTLs when you need to move traffic quickly (failover, deployments) at the cost of more lookups; use long TTLs for stable endpoints to reduce load and latency. Use GeoDNS when you serve a global audience from multiple regions.

Real-world examples

- **Cloudflare (1.1.1.1)** and **Google (8.8.8.8)** run huge public recursive resolvers.
- **Amazon Route 53** and **Cloudflare DNS** offer GeoDNS, health checks, and failover.
- When a site 'goes down' but is actually fine, the culprit is often a DNS misconfiguration or a stale cached record.

3. Load Balancers

What is it?

A load balancer sits in front of a group of servers and spreads incoming requests across them, so no single server is overwhelmed. To the outside world it looks like one address; behind it sit many identical workers.

Analogy: the person at the airport check-in hall who waves you to whichever desk is free. You see one queue; they route you to many counters.

Why does it exist?

One server can only handle so many requests before it slows down or crashes. To serve millions of users you run many servers — but users only know one web address. Something must decide which server handles each request, skip servers that have died, and keep traffic even. That something is the load balancer. It is also the foundation of **horizontal scaling** (adding more machines) and of **high availability** (if one server dies, traffic simply flows to the others).

How does it work internally?

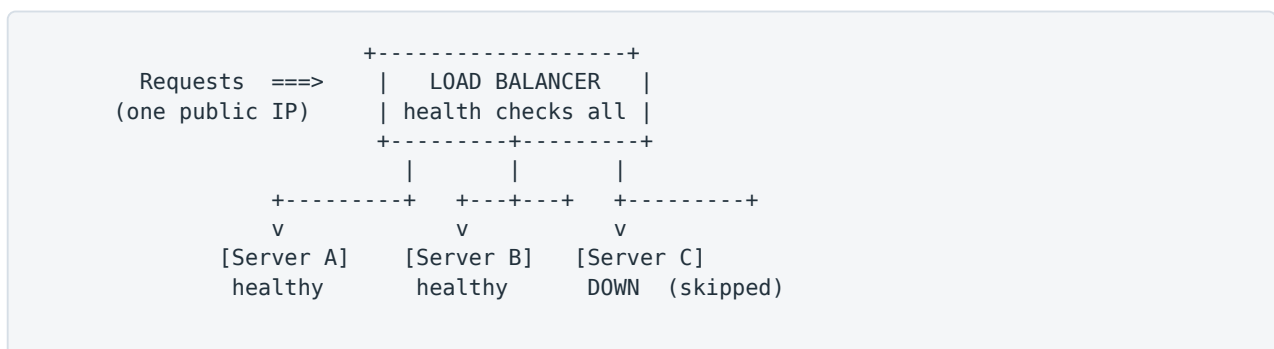


Figure 3.1 - The load balancer hides many servers behind one address and avoids dead ones.

The load balancer continuously runs **health checks** (e.g. "does /health return 200?"). Healthy servers stay in the pool; unhealthy ones are removed automatically and added back when they recover. This is what keeps a service online even as individual machines fail or are restarted during a deployment.

Layer 4 vs Layer 7

Load balancers operate at one of two levels of the networking stack:

- **Layer 4 (transport level)** works with TCP/UDP. It looks only at IP addresses and ports — not the content of the request. It is extremely fast and cheap because it just forwards packets, but it cannot make decisions based on, say, the URL.
- **Layer 7 (application level)** understands HTTP. It can read the URL, headers, and cookies, so it can route /images to one set of servers and /api to another, terminate HTTPS, and do sticky sessions. It is more flexible but does more work per request.

Aspect	Layer 4 (L4)	Layer 7 (L7)
Sees	IP + port only	Full HTTP: URL, headers, cookies
Speed	Very fast	Slightly slower (parses content)
Routing	By connection	By path, host, cookie, etc.
Use when	Raw throughput, any protocol	Smart HTTP routing, TLS termination

Balancing algorithms

How does the load balancer decide which server gets the next request? Common strategies:

- **Round robin:** hand requests to servers in turn — A, B, C, A, B, C. Simple and fair when all servers and requests are similar.
- **Weighted round robin:** give more powerful servers a bigger share (e.g. A gets 2 turns per cycle).
- **Least connections:** send the next request to whichever server currently has the fewest open connections. Better when requests vary a lot in how long they take.
- **Least response time:** pick the server that is both fast and lightly loaded.
- **IP hash / consistent hashing:** map a key (like the client IP) to a server so the same client keeps hitting the same server. Essential when servers hold per-user state or local caches.

Consistent hashing — the one to understand deeply

Plain hashing ($\text{server} = \text{hash}(\text{key}) \% N$) breaks badly when N changes: add or remove one server and almost every key maps somewhere new, wiping caches and reshuffling everything.

Consistent hashing solves this by placing servers and keys on an imaginary ring. Each key is owned by the next server clockwise. When a server is added or removed, only the keys in its slice of the ring move — everything else stays put.

Imagine a clock face (the hash ring, 0 -> max -> 0)

```

      Server B
      (12 o'clock)
        *
      key2 . . . key3
        .
Server A * . . . * Server C
(8 o'clock) . . . (4 o'clock)
        . key1 .

```

A key is stored on the FIRST server clockwise from it.
 Remove Server C -> only C's keys move (to the next server),
 not the entire keyspace.

Figure 3.2 - Consistent hashing limits how many keys move when servers change.

Interview tip

- Whenever you hear 'distributed cache', 'sharded database', or 'minimise reshuffling on scale-up', the expected answer almost always involves **consistent hashing** (often with virtual nodes to even out the distribution).

When should you use it?

Use a load balancer the moment you have more than one server — which, for any real service, is immediately, both for capacity and for survival when a machine fails. Choose L4 for raw speed and non-HTTP traffic; choose L7 when you need content-aware routing, TLS termination, or path-based splitting. In practice large systems layer them: a fast L4 layer at the edge feeding smart L7 balancers deeper in.

Real-world examples

- **Hardware/cloud:** AWS Elastic Load Balancing (NLB is L4, ALB is L7), Google Cloud Load Balancing.
- **Software:** NGINX, HAProxy, and Envoy are widely used as L7 load balancers and reverse proxies.
- **Inside data centres:** large companies build internal load balancers (e.g. Google's Maglev) to spread traffic across thousands of machines.

4. Web Servers vs Application Servers

What is it?

A **web server** is specialised at speaking HTTP and serving content — especially *static* files like HTML, images, CSS, and JavaScript. An **application server** runs your actual program logic — the code that decides what a logged-in user should see, processes a payment, or computes a feed. They are often used together.

Analogy: the web server is the front desk and display shelves of a shop (fast, organised, hands you ready-made items). The application server is the workshop in the back where custom orders are actually built.

Why does it exist?

Serving a fixed image and computing a personalised page are very different jobs. Serving files is simple and can be made blisteringly fast and memory-light. Running business logic needs a full runtime (Java, Python, Node, Go) with access to databases and other services. Separating the two lets each do what it is best at: the web server absorbs huge volumes of simple requests and shields the heavier application server, which can then focus on the dynamic work.

How does it work internally?

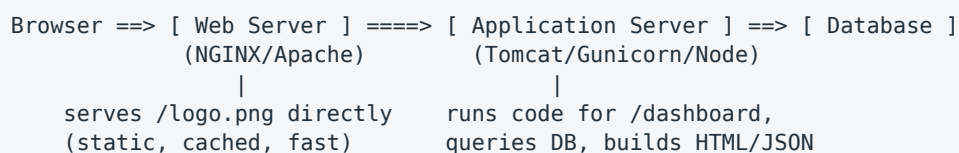


Figure 4.1 - The web server handles static content and forwards dynamic requests to the app server.

A typical flow: the web server receives every request. If the request is for a static file it owns, it returns it immediately. If the request needs computation ("show my dashboard"), the web server acts as a **reverse proxy** and forwards it to an application server, which runs the code, talks to databases and caches, builds the result, and hands it back. The web server can also compress responses, terminate HTTPS, and cache results — offloading work from the precious application tier.

When should you use it?

For anything beyond a toy, you run both. Let the web server handle static assets, TLS, compression, and basic routing; let application servers run logic and scale independently behind it. In modern cloud setups the 'web server' role is often filled by a load balancer/CDN combination, and the app tier is a fleet of containers — but the conceptual split between serving content and running logic remains.

Real-world examples

- **Web servers:** NGINX, Apache HTTP Server, Caddy.
- **Application servers / runtimes:** Apache Tomcat (Java), Gunicorn/uWSGI (Python), Node.js, Puma (Ruby), Kestrel (.NET).
- **Common pairing:** NGINX in front of Gunicorn running a Django app.

5. Databases — SQL vs NoSQL

What is it?

A database is an organised store of data that lets you reliably save information and get it back later, even with many users reading and writing at once. The two big families are **SQL (relational)** databases, which store data in tables with strict structure, and **NoSQL** databases, which trade some structure for flexibility and scale.

Why does it exist?

You could save data in plain files, but you would quickly need to answer questions like "find all orders over \$100 from last week", guarantee that money isn't lost when two people pay at once, and keep working when traffic explodes. Databases exist to provide **querying, integrity, concurrency control**, and **durability** — all the hard guarantees that raw files lack.

SQL (relational) databases

Data lives in **tables** of rows and columns with a fixed **schema** (you define the columns and their types up front). Tables relate to each other via keys, and you query with **SQL**, a declarative language. Their defining strength is **ACID** guarantees:

- **A — Atomicity**: a transaction happens fully or not at all. Transfer money and both the debit and credit succeed together, or neither does.
- **C — Consistency**: the database moves from one valid state to another, never breaking your defined rules (constraints).
- **I — Isolation**: concurrent transactions don't trip over each other; the result is as if they ran one after another.
- **D — Durability**: once committed, data survives crashes and power loss.

These guarantees make SQL databases the default for anything where correctness is non-negotiable: money, inventory, bookings.

NoSQL databases

NoSQL is an umbrella for databases that relax the rigid table model to gain flexibility and horizontal scale. The main types:

Type	Stores data as	Good for	Examples
Key-Value	A giant dictionary: key -> value	Caches, sessions, simple lookups	Redis, DynamoDB
Document	JSON-like documents	Flexible records, catalogues, profiles	MongoDB, Couchbase
Wide-Column	Rows with dynamic columns	Huge write volume, time-series	Cassandra, HBase

Type	Stores data as	Good for	Examples
Graph	Nodes and edges	Relationships, social graphs	Neo4j, Neptune

Many NoSQL systems favour **BASE** semantics instead of ACID:

- **BA — Basically Available:** the system stays responsive even under failure.
- **S — Soft state:** data may be in flux as it propagates between copies.
- **E — Eventual consistency:** given no new writes, all copies converge to the same value *eventually* — but a read right after a write might see stale data.

How does it work internally? (indexing & storage)

The single most important internal concept is the **index**. Without one, finding a row means scanning every row (a 'full table scan') — fine for 100 rows, catastrophic for 100 million. An index is a separate, sorted data structure that points to where rows live, turning a linear scan into a fast lookup.

```
Without an index: look at EVERY row to find user_id = 42
[ row1 ][ row2 ][ row3 ] ..... [ row 9,999,999 ] (slow!)

With a B-Tree index on user_id: follow a few pointers
      [50]
     /  \
    [25] [75]
   /  \ /  \
  [12] [42] [60] [90]  <- found 42 in ~log(N) steps (fast!)
```

Figure 5.1 - An index turns a slow full scan into a fast logarithmic lookup.

Two storage engines dominate. **B-Trees** (used by most SQL databases like MySQL/InnoDB and PostgreSQL) keep data sorted and are excellent for reads and range queries. **LSM-Trees** (Log-Structured Merge trees, used by Cassandra, RocksDB, and many write-heavy stores) buffer writes in memory and flush them in sorted batches, making writes extremely fast at some cost to reads. Knowing which engine a database uses tells you whether it is read-optimised or write-optimised.

Indexing trade-off

- Indexes make reads fast but make writes slower (every write must also update the indexes) and use extra storage. Index the columns you search and join on — not every column.

When should you use which?

- **Choose SQL** when data is structured and relationships matter, when you need transactions and strong consistency (finance, orders, inventory), and when you will run complex queries and joins.
- **Choose NoSQL** when you need massive horizontal scale, very high write throughput, flexible or rapidly-changing schemas, or a data shape that fits a specific model (a social graph, a time-series, a simple key-value cache).

- **In reality, large systems use both** (polyglot persistence): SQL for the money, a document store for the catalogue, a key-value store for sessions, and a wide-column store for analytics events.

Real-world examples

- **SQL**: PostgreSQL, MySQL, Microsoft SQL Server, Oracle; cloud: Amazon RDS, Google Cloud SQL.
- **NoSQL**: Redis (key-value), MongoDB (document), Cassandra (wide-column), Neo4j (graph), Amazon DynamoDB (managed key-value/document).

6. Caching

What is it?

A cache is a small, fast store that keeps copies of data you use often, so you can avoid the slow work of fetching or computing it again. Reading from a cache (memory) can be thousands of times faster than reading from a database on disk.

Analogy: keeping milk in your kitchen fridge (cache) instead of driving to the supermarket (database) every time you want a cup of tea. The fridge is small and fast; the shop is huge but slow.

Why does it exist?

Most systems read the same popular data over and over: the home page, a celebrity's profile, today's trending posts. Hitting the database for every one of those reads is wasteful and slow, and the database becomes a bottleneck. A cache absorbs the repeated reads, slashing latency for users and load on the database — often turning a struggling system into a comfortable one.

How does it work internally? (cache levels)

Caches exist at many layers, each closer-but-smaller than the last. The CPU itself has **L1/L2/L3** caches — tiny pools of ultra-fast memory next to the processor core that hold recently used values so the CPU rarely waits on slower main memory.

FASTER / SMALLER (closer to the work)		
+-----+		
CPU L1 cache	~1 ns	kilobytes
CPU L2 cache	~4 ns	hundreds of KB
CPU L3 cache	~10 ns	a few MB
Main memory (RAM)	~100 ns	gigabytes
In-memory store (Redis)	~0.5 ms over network	
SSD / Database disk	~1-10 ms	terabytes
Another data centre	~100 ms	
+-----+		
SLOWER / LARGER (farther away)		

Figure 6.1 - The memory hierarchy: each level trades size for speed. Caching exploits it.

In system design we mostly mean an **application-level cache** like Redis or Memcached — an in-memory store sitting between your app and the database. The app checks the cache first; only on a miss does it hit the database, then it stores the result for next time.

Caching strategies

How and when you write data into the cache defines its behaviour:

- **Cache-aside (lazy loading):** the app checks the cache; on a miss it reads the DB and populates the cache. Simple and common. Downside: the first read is slow, and stale data is possible until it expires.

- **Write-through:** every write goes to the cache *and* the database together. The cache is always fresh, but writes are a little slower.
- **Write-back (write-behind):** writes go to the cache immediately and are flushed to the database later in batches. Very fast writes, but risk of data loss if the cache dies before flushing.

```

CACHE-ASIDE (read path)
  app --> cache?  --HIT--> return value
                |
                MISS
                v
            read DB --> store in cache --> return value

```

Figure 6.2 - The cache-aside read path: check cache, fall back to DB, then populate.

Eviction policies

A cache is small, so when it fills up it must throw something out. The rule for choosing the victim is the **eviction policy**:

- **LRU (Least Recently Used):** evict whatever hasn't been touched for the longest. The sensible default — recent data tends to be reused.
- **LFU (Least Frequently Used):** evict whatever is accessed least often. Good when popularity is stable over time.
- **FIFO:** evict the oldest inserted item regardless of use.
- **TTL (Time To Live):** every item expires after a set time, guaranteeing data is never older than that.

Watch out for these cache hazards

- **Cache stampede:** a popular key expires and thousands of requests hit the DB at once. Fix with locks or staggered expiry.
- **Stale data:** the cache and DB disagree after an update. Decide your freshness needs and invalidate carefully — 'cache invalidation' is famously one of the hard problems.

When should you use it?

Cache when reads vastly outnumber writes, the same data is requested repeatedly, and slightly stale data is acceptable. Do not cache data that must be perfectly up-to-date every time (a live bank balance), or data so unique per request that you would never get a cache hit.

Real-world examples

- **Redis:** in-memory data store, also supports rich data types, persistence, and pub/sub.
- **Memcached:** a simpler, pure in-memory key-value cache, great for raw speed.
- **Browser & CDN caches:** store assets close to users (covered next).
- Twitter, Facebook, and almost every large site put Redis/Memcached in front of their databases.

7. Content Delivery Networks (CDN)

What is it?

A CDN is a worldwide network of servers (called **edge servers** or **PoPs** — points of presence) that store copies of your content close to your users. When someone in Tokyo requests your logo, they get it from a nearby Tokyo edge server instead of a data centre in Virginia.

Analogy: a global chain of local warehouses. Instead of shipping every order from one central factory, popular goods are pre-stocked in a warehouse near each city for same-day delivery.

Why does it exist?

Distance costs time. Data travels near the speed of light, but a round trip from Australia to a US server and back still adds hundreds of milliseconds — enough to make a site feel sluggish. CDNs cut that distance dramatically, speeding up load times, reducing load on your origin servers, absorbing traffic spikes, and helping defend against attacks.

How does it work internally?

	Without a CDN	With a CDN
User(JP)	-----> Origin(US)	--> Edge(Tokyo) [cached]
	long round trip	short hop, fast
		(Edge fetches from Origin only on a miss, then caches it)

Figure 7.1 - A CDN serves cached content from the edge nearest the user.

When a user requests a file, DNS (often GeoDNS) points them at the nearest edge server. If that edge already has the file cached (a **cache hit**), it serves it instantly. If not (a **cache miss**), the edge fetches it once from your **origin** server, stores it, and serves all future requests from the cache until it expires. CDNs are excellent for **static** content (images, video, CSS, JS) and increasingly handle **dynamic** content and even run code at the edge.

When should you use it?

Use a CDN whenever you serve large static assets, have a geographically spread audience, stream video, or need to survive traffic spikes. There is little reason *not* to put a CDN in front of static content for any public site. Be careful with content that changes constantly or must be personalised per user — you cache those differently (short TTLs, cache keys per user, or not at all).

Real-world examples

- **Cloudflare, Akamai, Fastly, Amazon CloudFront, Google Cloud CDN** are major providers.

- Netflix runs its own CDN (Open Connect), placing video caches inside ISPs to serve streams locally.
- Nearly every image, video, and script on big websites is delivered via a CDN.

8. Message Queues & Event Streaming

What is it?

A message queue lets one part of a system send a message and move on, while another part picks it up and processes it later. The sender (**producer**) and the receiver (**consumer**) never talk directly; the queue sits in the middle holding messages. This is **asynchronous** communication — 'fire and forget, process later'.

Analogy: a restaurant's order rail. Waiters clip orders to the rail (produce) and keep serving; cooks take orders off the rail when free (consume). Neither has to wait for the other.

Why does it exist?

Some work shouldn't block the user. When you upload a video, you don't want to stare at a spinner while it is transcoded into five resolutions — the app should say 'uploaded!' instantly and do the heavy work in the background. Queues provide this, and three more superpowers: **decoupling** (services don't need to know about each other), **buffering** (absorbing traffic spikes so a slow consumer doesn't crash), and **resilience** (if a consumer is down, messages wait safely instead of being lost).

How does it work internally?

```
[ Producer ] --put--> [  QUEUE  ] --take--> [ Consumer(s) ]
(web app)           | msg | msg |          (workers)
                   | msg | msg |
```

Spikes are absorbed here; consumers process at their own pace.
If a worker dies, its message is re-delivered to another worker.

Figure 8.1 - A queue decouples producers from consumers and buffers bursts.

Two messaging patterns

- **Point-to-point (work queue):** each message is processed by exactly *one* consumer. Used to distribute tasks among a pool of workers (e.g. send-email jobs). RabbitMQ excels here.
- **Publish/Subscribe (pub/sub):** each message is delivered to every interested subscriber. One 'order placed' event can simultaneously trigger billing, shipping, and analytics. Kafka and cloud pub/sub services shine here.

```
PUB/SUB
                                +--> [ Billing service ]
[ Publisher ] --> TOPIC --> [ Shipping service ]
"OrderPlaced"      +--> [ Analytics service ]
One event, many independent subscribers each get a copy.
```

Figure 8.2 - Pub/sub fans one event out to many subscribers.

Message queues vs event streaming

A traditional **message queue** (RabbitMQ) deletes a message once it has been processed — it is a to-do list. An **event streaming platform** (Kafka) keeps an ordered, durable **log** of events that can be re-read by many consumers and replayed from any point — it is a permanent ledger. Kafka can handle enormous throughput and is the backbone of real-time data pipelines, while RabbitMQ is a flexible, feature-rich broker for task distribution and routing.

Aspect	Message Queue (RabbitMQ)	Event Stream (Kafka)
Model	Messages consumed and removed	Append-only durable log
Replay	Generally no	Yes — re-read from any offset
Throughput	High	Very high (millions/sec)
Best for	Task queues, routing, RPC	Event sourcing, analytics, pipelines

When should you use it?

Use a queue whenever work can be done later, when you want to smooth out spikes, or when you want to decouple services so they can fail and scale independently. Reach for streaming (Kafka) when you need to process high-volume event data, want multiple independent consumers, or need to replay history. Avoid queues for work that must complete *before* you can answer the user (synchronous requests).

Real-world examples

- **Kafka**: LinkedIn (its birthplace), Uber, and Netflix use it for real-time event pipelines.
- **RabbitMQ**: widely used for background jobs and microservice messaging.
- **Cloud**: Amazon SQS/SNS, Google Pub/Sub, Azure Service Bus.
- Classic uses: sending emails, processing uploads, order processing, notifications, analytics ingestion.

9. APIs — REST, GraphQL, gRPC

What is it?

An API (Application Programming Interface) is the agreed set of rules for how two pieces of software talk to each other. It is the menu and the ordering process at a restaurant: it tells the client what it can ask for and how to ask, while hiding the kitchen's inner workings.

Why does it exist?

Systems are built from many parts — a mobile app, a website, dozens of backend services, and third-party partners. They need a stable, documented contract to communicate without knowing each other's internals. APIs provide that contract, so teams can build and change their services independently as long as they honour the agreed interface.

The three main styles

REST

REST treats everything as a **resource** with a URL, manipulated using standard HTTP verbs: GET to read, POST to create, PUT/PATCH to update, DELETE to remove. It usually exchanges JSON. REST is simple, universal, cacheable, and human-readable — the default choice for public web APIs. Its weaknesses: you can **over-fetch** (get more fields than you need) or **under-fetch** (need several requests to assemble one screen).

```
GET    /users/42           -> read user 42
GET    /users/42/orders   -> read user 42's orders
POST   /users             -> create a new user
DELETE /users/42          -> delete user 42
```

Figure 9.1 - REST maps HTTP verbs onto resource URLs.

GraphQL

GraphQL exposes a single endpoint where the **client specifies exactly what data it wants** in a query, and gets back precisely that — no more, no less. This eliminates over- and under-fetching and is ideal for rich front-ends that assemble many pieces of data. The cost: it is harder to cache (everything is a POST to one URL), and a careless client can write an expensive query.

```
query {
  user(id: 42) {
    name
    orders { id, total }  <- ask for just these fields,
                        in one round trip
  }
}
```

Figure 9.2 - A GraphQL query fetches exactly the fields requested.

gRPC

gRPC is a high-performance method of calling functions on a remote server as if they were local. It uses **Protocol Buffers** (a compact binary format) over HTTP/2, making it very fast and efficient, with strongly-typed contracts and built-in streaming. It is the favourite for **internal service-to-service** communication in microservices. The trade-off: the binary format is not human-readable and is awkward to call directly from browsers.

Choosing between them

	REST	GraphQL	gRPC
Format	JSON (text)	JSON (text)	Protobuf (binary)
Best for	Public web APIs	Rich, varied front-ends	Internal microservices
Speed	Good	Good	Excellent
Fetching	Over/under-fetch	Exact fields	Exact (typed)
Caching	Easy (HTTP)	Harder	Manual

When should you use it?

- **REST**: public APIs, simple CRUD services, anything that benefits from HTTP caching and broad tooling. The safe default.
- **GraphQL**: mobile and web clients that need flexible, tailored data and want to avoid many round trips.
- **gRPC**: low-latency, high-throughput communication between your own backend services, or streaming use cases.

Real-world examples

- **REST**: the Stripe, Twitter, and GitHub public APIs.
- **GraphQL**: invented at Facebook; used by GitHub's v4 API, Shopify, and many app back-ends.
- **gRPC**: heavily used inside Google, and across Kubernetes and cloud-native infrastructure.

10. Proxies — Forward vs Reverse

What is it?

A proxy is a middleman that sits between a client and a server and relays traffic between them. The direction it faces gives it its name: a **forward proxy** acts on behalf of *clients*; a **reverse proxy** acts on behalf of *servers*.

Forward proxy

A forward proxy sits in front of clients and forwards their requests out to the internet. The destination server sees the proxy, not the original client. Companies use them to filter and monitor employee browsing; individuals use them for privacy or to bypass regional restrictions.

```
[ Client ] --> [ Forward Proxy ] --> ( Internet ) --> [ Server ]  
Hides/represents the CLIENT. Server sees the proxy's address.  
Uses: content filtering, caching, anonymity, access control.
```

Figure 10.1 - A forward proxy represents clients to the wider internet.

Reverse proxy

A reverse proxy sits in front of your servers and receives requests on their behalf, then forwards them to the right internal server. Clients only ever see the proxy. This is one of the most useful tools in system design: a single front door that adds security and capabilities.

```
[ Client ] --> [ Reverse Proxy ] --> [ Server A ]  
                        |               +-----> [ Server B ]  
Hides/represents the SERVERS. Client sees only the proxy.  
Does: load balancing, TLS termination, caching, compression,  
      hiding internal structure, blocking bad traffic.
```

Figure 10.2 - A reverse proxy is the single front door to many backend servers.

If this sounds like a load balancer — it is closely related. A load balancer is a specialised reverse proxy focused on distributing traffic. Reverse proxies like NGINX commonly do load balancing *and* TLS termination, caching, and security filtering all at once.

When should you use it?

Use a **reverse proxy** in front of essentially every production web service — it centralises TLS, caching, compression, and security, and hides your internal layout. Use a **forward proxy** when you need to control, monitor, or anonymise outbound traffic from a set of clients (corporate networks, web scraping, privacy tools).

Real-world examples

- **Reverse proxies:** NGINX, HAProxy, Envoy, and CDNs like Cloudflare (which is a giant reverse proxy in front of millions of sites).
- **Forward proxies:** corporate Squid proxies, VPNs (related idea), privacy proxies.

11. Storage Systems — Block, File, Object

What is it?

Storage is where your data physically rests. There are three ways to organise raw storage, each exposing data differently: **block**, **file**, and **object** storage. They are not competitors so much as tools for different jobs.

The three types

Block storage

Storage is carved into fixed-size **blocks**, like raw building bricks, with no built-in notion of files. An operating system or database formats these blocks into whatever structure it likes. Block storage is extremely fast and low-level — it is what you attach to a server as a 'hard drive'. Ideal for databases and operating systems that need fine-grained, high-performance access.

File storage

Storage is organised the familiar way: **files** inside **folders** in a hierarchy, accessed by path (/projects/report.docx). It is intuitive and great for shared documents and content that humans and applications browse by path. It becomes harder to scale to billions of files than object storage.

Object storage

Each piece of data is stored as an **object** — the data itself, plus rich metadata, plus a unique ID — in a flat namespace (a 'bucket'), accessed over HTTP APIs. There is no folder tree; you fetch an object by its key. Object storage scales almost limitlessly and cheaply, making it the default home for images, videos, backups, and big-data files. Trade-off: you usually replace whole objects rather than editing them in place, and latency is higher than block storage.

BLOCK	raw bricks -> OS/DB builds its own structure (fast, low-level) [blk][blk][blk] ... e.g. AWS EBS, a virtual hard disk
FILE	folders and paths (familiar, shared) /home/docs/report.txt e.g. NFS, AWS EFS, a network drive
OBJECT	flat bucket of {data + metadata + key} (vast, cheap, web) bucket/photos/cat.jpg e.g. Amazon S3, Google Cloud Storage

Figure 11.1 - Three storage models, from low-level bricks to web-scale buckets.

How does it work internally? (distributed storage)

At large scale, storage is spread across many machines for capacity and safety. **HDFS** (the Hadoop Distributed File System) splits a huge file into large blocks and stores multiple copies of each block on different machines, so the file survives disk failures and can be processed in parallel. Object stores like **S3** similarly replicate objects across multiple devices and data centres, which is why they offer such high durability (Amazon advertises 99.999999999% — 'eleven nines').

When should you use which?

- **Block:** databases, operating system disks, anything needing fast low-level read/write to attached storage.
- **File:** shared file systems, content many applications open by path, legacy apps expecting a filesystem.
- **Object:** images, video, backups, logs, static website assets, data-lake files — anything large, numerous, and accessed over the web.

Real-world examples

- **Block:** Amazon EBS, Google Persistent Disk.
- **File:** NFS, Amazon EFS, Azure Files.
- **Object:** Amazon S3, Google Cloud Storage, Azure Blob Storage; **HDFS** for big-data clusters.

12. Networking Basics

What is it?

Networking is how computers actually exchange data. You don't need to be a network engineer, but a handful of concepts come up in almost every system design discussion: TCP vs UDP, HTTP/HTTPS, and the ways a client and server can keep a live conversation going.

TCP vs UDP

Both are ways to send data across the internet, but they make opposite trade-offs.

- **TCP (Transmission Control Protocol)** is reliable. It establishes a connection (a 'handshake'), guarantees that data arrives in order and without loss (re-sending anything dropped), and controls its sending rate. The cost is extra overhead and latency. Used for the web, email, file transfer — anywhere correctness matters.
- **UDP (User Datagram Protocol)** is fast and lightweight. It just fires packets off with no handshake and no guarantee they arrive or arrive in order. The cost is possible loss; the benefit is speed. Used for live video, voice calls, and online games, where a slightly dropped frame matters less than staying real-time.

```
TCP: "Did you get packet 1?" "Yes." "Packet 2?" "Yes." ...  
      reliable, ordered, slower    (a careful phone call)  
  
UDP: "Here's 1! Here's 2! Here's 3!" (no confirmation)  
      fast, may lose some          (shouting across a room)
```

Figure 12.1 - TCP guarantees delivery; UDP trades guarantees for speed.

HTTP and HTTPS

HTTP is the request/response protocol of the web, built on top of TCP. **HTTPS** is HTTP wrapped in encryption (TLS), so that data travelling between client and server cannot be read or tampered with along the way. HTTPS also verifies you are really talking to the site you think you are, using certificates. Today HTTPS is the default for everything — never send sensitive data over plain HTTP.

Keeping a live connection: polling, long polling, and WebSockets

Classic HTTP is one-shot: the client asks, the server answers, done. But many apps need the server to push updates to the client — new chat messages, live scores, notifications. Three approaches, in increasing sophistication:

- **Short polling:** the client repeatedly asks 'anything new?' every few seconds. Simple but wasteful — most answers are 'no', burning requests.

- **Long polling:** the client asks, and the server *holds the request open* until it has something to send (or a timeout), then the client immediately asks again. Far fewer wasted requests, near-real-time, and works everywhere — but still one direction at a time.
- **WebSockets:** opens a single, *persistent, two-way* connection. Either side can send messages at any time with low overhead. The right tool for true real-time apps: chat, multiplayer games, live dashboards, collaborative editors.

```
SHORT POLL  client: any news? (no) ... any news? (no) ... (yes!)
LONG POLL   client: any news? --> [server holds] --> (yes!) --> ask again
WEBSOCKET   client <===== open 2-way channel =====> server
              both sides push messages freely, anytime
```

Figure 12.2 - From wasteful polling to an always-open WebSocket channel.

When should you use which?

- **TCP / HTTP(S):** the default for almost all request/response traffic.
- **UDP:** real-time media and gaming where speed beats perfect reliability.
- **Long polling:** simple near-real-time updates with broad compatibility and minimal infrastructure.
- **WebSockets:** high-frequency, bidirectional, low-latency communication.

Real-world examples

- **WebSockets:** WhatsApp Web, Slack, online multiplayer games, live trading dashboards.
- **UDP:** Zoom/Google Meet media streams, fast-paced games, DNS itself.
- **HTTPS:** every reputable website you visit today.

Phase 1 — Key Takeaways

Your one-page revision sheet. If you can explain each line below in your own words, you have mastered Phase 1.

Core Components — the essentials

- **Client/Server:** one side requests, the other responds; keep trust and shared data on the server.
- **DNS:** the internet's phone book; turns names into IPs, and doubles as a global routing tool.
- **Load Balancer:** spreads traffic across many servers; L4 is fast and dumb, L7 is content-aware; consistent hashing minimises reshuffling.
- **Web vs App Server:** web servers serve static content fast; app servers run your logic.
- **SQL vs NoSQL:** SQL gives ACID and structure (use for money/orders); NoSQL gives scale and flexibility (BASE, eventual consistency). Indexes turn slow scans into fast lookups.
- **Caching:** keep hot data in fast memory; know cache-aside / write-through / write-back and eviction policies (LRU). Beware stale data and stampedes.

Core Components — the rest

- **CDN:** serve static content from edge servers near users; cuts latency and origin load.
- **Queues & Streaming:** decouple and buffer with async messaging; RabbitMQ = task queue, Kafka = durable replayable log; point-to-point vs pub/sub.
- **APIs:** REST (default, cacheable), GraphQL (exact data, flexible front-ends), gRPC (fast binary, internal services).
- **Proxies:** forward proxy represents clients; reverse proxy represents servers and is the front door for TLS, caching, and load balancing.
- **Storage:** block (fast, low-level), file (paths/folders), object (vast, cheap, web — S3).
- **Networking:** TCP reliable vs UDP fast; HTTPS always; WebSockets for real-time two-way; long polling for simple near-real-time.

What comes next

- In **Phase 2** we learn the principles that combine these components into reliable, scalable systems: scalability, availability, the CAP theorem, consistency models, sharding, replication, rate limiting, and security.